Smart Virtual Assistant - Proposal Project

---

1. The Ill-Defined Problem

1.1 Problem Statement

Students who recently moved to the city or are unfamiliar with local routes often face difficulties in finding food vendors that match their budget and taste preferences. This challenge is especially prominent when they have not yet built a network with local friends or residents who can assist them in navigating the area.

1.2 Ambiguous & Edge Test Cases

This section outlines ambiguous user inputs, qualitative criteria, or edge cases that require clarification, threshold handling, or default fallback mechanisms from the system:

- **Test Case 1 (Irrelevant Input):**
    - *Input:* User inputs irrelevant or non-food-related information.
    - *Expected Output:* Prompt the user to re-enter input or display an error message.
- **Test Case 2 (Missing Location):**
    - *Input:* User does not provide their current location.
    - *Expected Output:* Prompt the user to request location access/input.
- **Test Case 3 (Food Safety & Quality Assessment):**
    - *Input:* A restaurant/vendor offers lower prices, but its quality and food safety standards are not guaranteed compared to other options.
    - *Expected Output:* Assign a low ranking/priority score to the vendor.

---

2. The Well-Defined Problem

This section outlines deterministic scenarios with clear inputs, structured rules, and explicit logic:

- **Test Case 1 (Budget-Oriented Search):**
    - *Input:* User provides only maximum budget/spending limit.

    - *Expected Output:* Return top-rated vendors sorted by lowest price (Prioritize Price).
- **Test Case 2 (Dish-Oriented Search):**
    - *Input:* User provides only the desired dish name.
    - *Expected Output:* Return vendors serving that dish, starting from the lowest price with high rating scores (Prioritize Rating).
- **Test Case 3 (Cost vs. Distance Optimization - Fuel Cost Exceeds Price Savings):**
    - *Input:* A vendor is cheaper and higher rated, but located significantly further away such that the estimated fuel cost exceeds the price difference compared to closer options.
    - *Expected Output:* Recommend the closer vendor option.

---

3. The Four Pillars of Computational Thinking

3.1 Overview of the Four Pillars

Pillar	Core Meaning	Primary Question
Decomposition	Breaking a complex problem into smaller, manageable sub-problems.	How can this problem be divided?
Pattern Recognition	Identifying trends, similarities, and repeatable conditions.	What patterns or common behaviors exist?
Abstraction	Filtering out unnecessary details to focus on critical data.	Which information is strictly essential?
Algorithm Design	Developing step-by-step procedures to reach a solution.	What sequence of actions must be taken?

3.2 Decomposition

Core Objective

To divide the overall objective—*helping students find suitable food vendors based on budget, food type, safety, and distance*—into modular tasks.

Module Breakdown

Module	Key Tasks & Responsibilities
A. Need Identification	Capture user budget, dish preference, taste, allergies, location, and distance limit.
B. Data Management	Store vendor name, menu items, prices, location coordinates, hours, ratings, and safety alerts.
C. Search & Ranking	Query candidate vendors, filter hard constraints, and score relative to distance, price, and quality.
D. Navigation	Identify start/destination points, compute routes, estimate travel time/costs, and guide through alleys.
E. Food Safety & Allergies	Track safety reports, issue warnings, manage allergen profiles, and evaluate risk factors.
F. Feedback & Review	Store user ratings, manage report logs, flag anomaly reviews, and bookmark vendors.

Input, Processing, Output Decomposition

- **Input:** Dish preference, budget constraint, user geolocation, allergy constraints, transit mode.
- **Processing:** Input Validation → Search Candidate Vendors → Filter Constraints → Evaluate Risk → Calculate Total Cost → Rank Candidates.
- **Output:** Ranked vendor list, cost breakdown, warnings, distance, and navigation path.

3.3 Pattern Recognition

User Request Patterns

Pattern Type	Input Example	Processing Strategy
Budget Only	"I have 30,000 VNĐ for lunch."	Treat price as primary filter/ranking key; differentiate using ratings.
Dish Only	"I want Pho."	Filter vendors offering the dish; prioritize ratings and verified food safety.
Budget & Dish	"Rice dishes under 35,000 VNĐ."	Enforce strict dual-constraint filtering, then rank remaining candidates.

Price vs. Travel Cost Trade-off Pattern

Let ΔP be food savings at a cheaper vendor, and ΔT be the additional travel cost.

- **Pattern 1 ($\Delta T > \Delta P$):** Travel costs exceed food savings → Recommend closer option.
- **Pattern 2 ($\Delta T = \Delta P$):** Financial costs are equal → Break tie using ratings, travel time, or user preferences.
- **Pattern 3 ($\Delta T < \Delta P$):** Total cost decreases → Cheaper distant vendor is viable.

Anomaly Review Patterns

- Spike in review count within an unnaturally short duration.
- High textual similarity across multiple distinct accounts.
- Rapid rating score fluctuations without verifiable operational changes.

3.4 Abstraction

Essential Attribute Modeling

Focus solely on features necessary for mathematical decision-making:

- **Vendor Profile:** `$\text{VendorID}, \text{Coordinates}, \text{PriceRange}, \text{VerifiedStatus}, \text{OperatingHours}$`.

- **User Profile:** $\text{UserLocation}, \text{MaxBudget}, \text{Allergies}, \text{TargetDish}$.
- **Trip Cost Abstraction:** $$C_{\text{total}} = C_{\text{food}} + C_{\text{travel}}$$

Contextual Recommendation Score Model

Conceptual representation for multi-factor candidate scoring:

$$S(u, r) = f(P, Q, D, T, R, A)$$

Where:

- P: Price fit
- Q: Dish relevancy
- D: Distance/travel cost
- T: Estimated travel time
- R: Review score and confidence weight
- A: Safety/Allergen verification

3.5 Algorithm Design

Main Request Handling Workflow (Pseudocode)

```
None
FUNCTION ProcessUserRequest(request):
    IF request is empty:
        Prompt for budget or dish preference
        RETURN
    IF request contains invalid/irrelevant data:
        Return Error ("Invalid Request")
        RETURN
    IF location is required AND location IS NULL:
        Prompt user to grant/input geolocation
        RETURN

    candidates = SearchVendors(request.dish, request.budget)
    filtered = FilterSafetyAndAllergens(candidates, request.allergies)
    ranked = RankCandidatesByTotalCostAndRating(filtered, request.location)

    RETURN ranked
```

Ranking & Filtering Logic

1. **Hard Filtering:** Eliminate vendors exceeding strict budget limits, failing critical safety protocols, or containing specified allergens.
2. **Cost Calculation:** Compute $C_{\text{total}} = C_{\text{food}} + C_{\text{travel}}$ for all remaining candidates.
3. **Scoring Function:** Normalize parameters and score candidates:

$$S(r) = w_P \cdot P_r + w_Q \cdot Q_r + w_D \cdot D_r + w_R \cdot R_r$$

4. **Tie-Breaking:** If two vendors have equal score, prioritize the one closer to the user location.

---

4. Solutions and Exploration Directions

4.1 System Interaction Pipeline (7-Step Architecture)

None

```
[ User Request ]
       |
       ▼
1. REQUEST         —— Receives text/image via Web/Mobile Chatbot UI
(HTTPS/WebSocket)
       |
       ▼
2. UNDERSTAND      —— Parses intent, extracts entities (Location, Budget, Group
Size, Tags)
       |
       ▼
3. ROUTE           —— Directs payload to specialized modules (Budget Engine,
Navigation, QA)
       |
       ▼
4. RETRIEVE/COMPUTE—— Queries Vector/Relational DB; calculates max per-person
budget & distance
       |
       ▼
5. REASON          —— LLM/Reasoning Engine synthesizes evidence, selects optimal
option
       |
       ▼
6. VERIFY          —— Guardrails check constraints (Budget <= Max, Open hours,
Safety checks)
       |
       ▼
7. RESPOND/ACT     —— Renders formatted cards/maps on Chat UI; logs operations
```

4.2 Case Study Execution Walkthrough

Scenario Setup

- **User Context:** A group of 3 HCMUS students at Campus 2 (Linh Trung - Di An) are searching for a quiet milk tea spot to study on a weekday afternoon, with a total combined budget of 100,000 VNĐ.

Step-by-Step Pipeline Execution

1. REQUEST

- **User Query:** *"There are 3 of us with a total budget of 100k. We want to find a quiet milk tea spot to study near Campus 2. Please give us easy step-by-step alley directions as well!"*

2. UNDERSTAND

NLU Parser extracts structured JSON:

```json
{
  "intent": "Search_Hidden_Cafe_With_Navigation",
  "constraints": {
    "location": "CS2_LinhTrung",
    "people_count": 3,
    "total_budget": 100000,
    "purpose": "study_quiet",
    "item_type": "milk_tea",
    "requires_alley_navigation": true
  }
}
```

3. ROUTE

Router dispatches request in parallel to:

- **Budget Optimization Engine** (Item/Price combination matching)
- **Micro-Navigation Retrieval Module** (Alley landmark sequence extraction)

4. RETRIEVE / COMPUTE

- **Calculation:** $$\text{Budget Per Person} = \frac{100,000\text{ VNĐ}}{3} \approx 33,333\text{ VNĐ/person}$$
- **Query:** Search candidates where `location == CS2_LinhTrung`, `tags CONTAINS ["quiet", "power_outlets"]`, and `item_price <= 33000`.
- **Retrieved Candidate: "Tra Tro Cafe & Milk Tea"**
  - *Beverage Price:* $25,000 - 30,000\text{ VNĐ}$ per item.
  - *Parking Fee:* 0 VNĐ (Free student parking).
  - *Landmarks:* `[Dorm Gate A] -> [Alley 12] -> [Yellow Utility Pole] -> [Cat Cat Laundry] -> [Destination]`.

5. REASON

- Selected 3 standard milk teas at $28,000\text{ VNĐ}$ each: $$\text{Total Cost} = 3 \times 28,000\text{ VNĐ} = 84,000\text{ VNĐ} \quad (\le 100,000\text{ VNĐ})$$
- Formatted detailed landmark navigation instructions.

6. VERIFY

- ✅ **Operating Hours Check:** Open ($08:00 - 22:00$), current time is $14:30$ → **PASS**.
- ✅ **Budget Constraint Check:** $84,000\text{ VNĐ} \le 100,000\text{ VNĐ} \rightarrow$ **PASS**.
- ✅ **Navigation Integrity Check:** Complete landmark sequence confirmed → **PASS**.

7. RESPOND / ACT

The system returns the formatted output card to the student UI:

> **HCMUS Virtual Assistant Response:**
>
> ☕ **Recommended Venue: Tra Tro - Cafe & Milk Tea**
>
> - **Environment:** Quiet, spacious tables, equipped with power outlets for laptops.
>
> 💰 **Estimated Cost Breakdown:**
>
> - Traditional Milk Tea: $28,000\text{ VNĐ} \times 3 = 84,000\text{ VNĐ}$
> - Parking Fee: Free (0 VNĐ)
> - *Remaining Savings:* $16,000\text{ VNĐ}$ (out of 100,000 VNĐ budget)
>
> 🗺️ **Alley Navigation Steps:**
>
> 1. From **Dorm Gate A (KTX ĐHQG)**, walk straight for 100m.

2. **Turn right** into Alley 12 (opposite Com Tam eatery).
3. Walk 30m to the **yellow utility pole**, then turn left at **Cat Cat Laundry**.
4. Destination is at the end of the lane on your left (look for the wooden sign "Tra Tro").